



Table of Contents

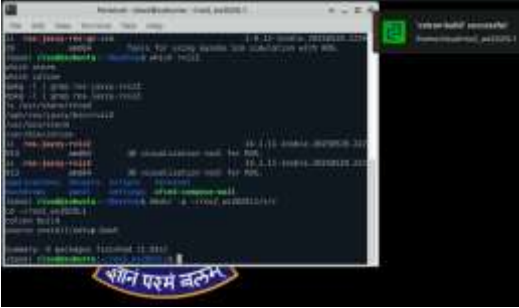
References	1
Lab 1: Implement Odometry based Motion Control for Obstacle Avoidance in ROS2+RViz.....	2
Appendix I	9
Appendix II	9
Self-Practice Exercises for the students.....	17

References

1. Getting Started with ROS ([Click here to learn about environment , topics, nodes, services, etc.,](#)).
2. Students are advised to use the BITS VM machines login to try the experiment [available in this page](#) before trying the webinar exercises.
Important Note: Replace the ROS2 version from “foxy” to “jazzy”

Lab 1: Implement Odometry based Motion Control for Obstacle Avoidance in ROS2+RViz

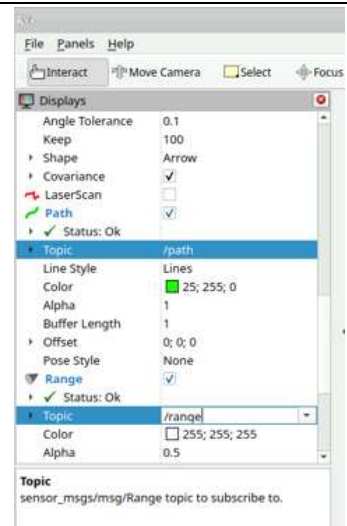
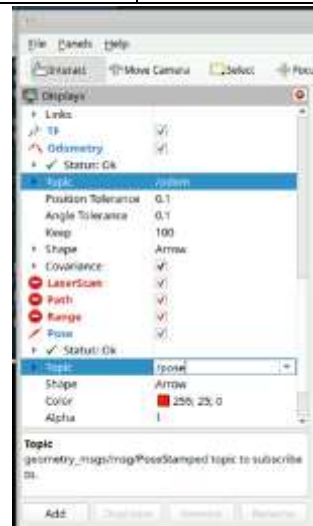
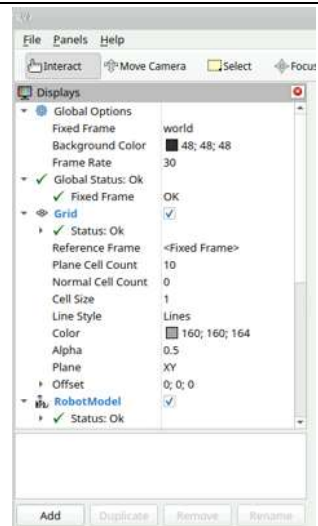
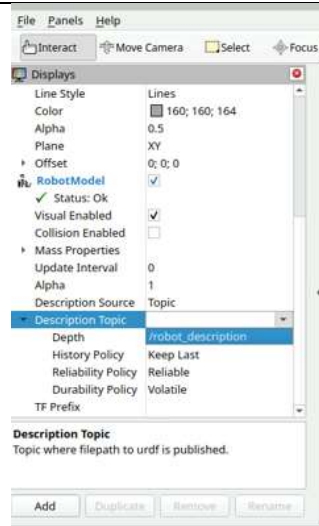
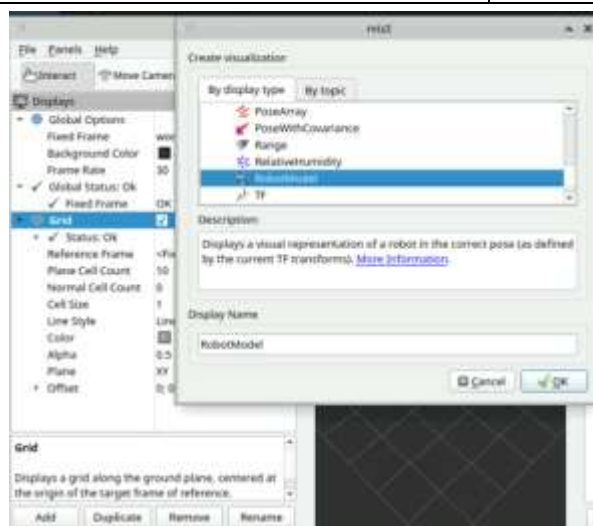
Refer to the [Appendix I](#) for the details on the architecture designed for this experiment

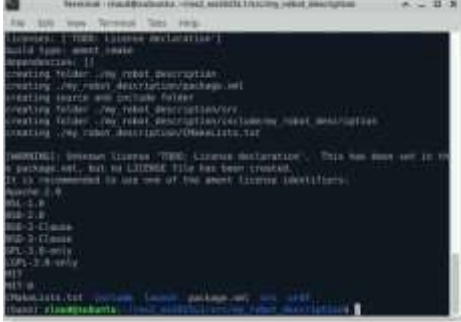
Lab 1	Objective : This comprehensive guide should get you started with mimicking traditional odometer based motion control in ROS2 and RViz using Python		
I	Environment & Package Set up		
	Instruction	Commands to Execute	Note/Output
1.1	Deactivate the conda environment	conda deactivate	
1.2	Setup your ROS2 workspace	mkdir -p ~/ros2_ws2025L1/src cd ~/ros2_ws2025L1 colcon build source install/setup.bash	
1.3	Create a ROS2 Package	cd ~/ros2_ws/src	
1.4	Create the ROS2 package for the project	cd ~/ros2_ws2025L1/src ros2 pkg create --build-type ament_cmake my_robot_description	
	Wait for some time till the folder creates successfully		
	Configure the Simulator		
1.5	Open another terminal and run RVIZ	rviz	This opens the RVIZ simulator with empty view
1.6	Once the simulator opens from previous step, preliminary set-ups must be checked/corrected to get a neat visualization. From the left navigation panel all the option to set up will be available. The project will not be visible until all the topics are set up to subscribe the data for visualization!	1. Open the “Display” panel on the left of the simulation window 2. Change the “Fixed Frame” under “Global Options” to “world” 3. Click the “Add” button at the bottom left 4. Add each below options and hardcode the topic label under “Topic” • RobotModel Click Add → RobotModel Set Description Topic → /robot_description • Odometry Click Add → Odometry Set Topic → /odom Adjust Shape → Arrow • Pose Click Add → Pose Set Topic → /pose	Once done save the configuration from the menu option in the RVIZ tool!

Prepared by,
Prof. Raja vadhana
raja.vadhana@pilani.bits-pilani.ac.in
Course Faculty

Updated & Reviewed by,
Prof. Raja vadhana, Prof. Sangeetha
raja.vadhana@pilani.bits-pilani.ac.in , sangeetha.viswanathan@pilani.bits-pilani.ac.in
Course Faculty

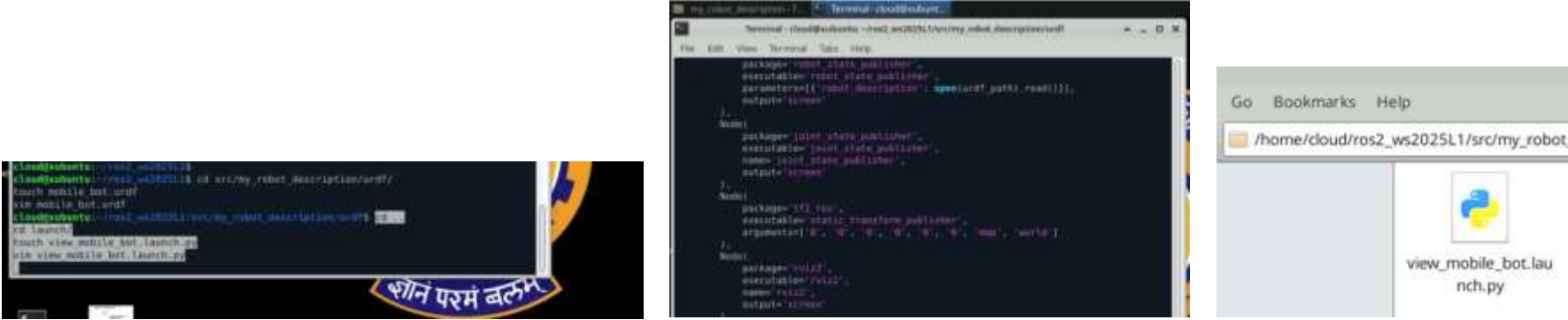
- Shape → Axes or Arrow
- Path
 - Click Add → Path
 - Set Topic → /path
 - Color → choose a bright color (e.g. yellow) for visibility
- Range (feature)
 - Display: Range
 - Topic: /range
 - Configure color/alpha as desired
- LaserScan
 - Click Add → LaserScan
 - Set Topic → /scan
 - Style → Points or Flat Squares
 - Adjust Size → 0.05 for clarity

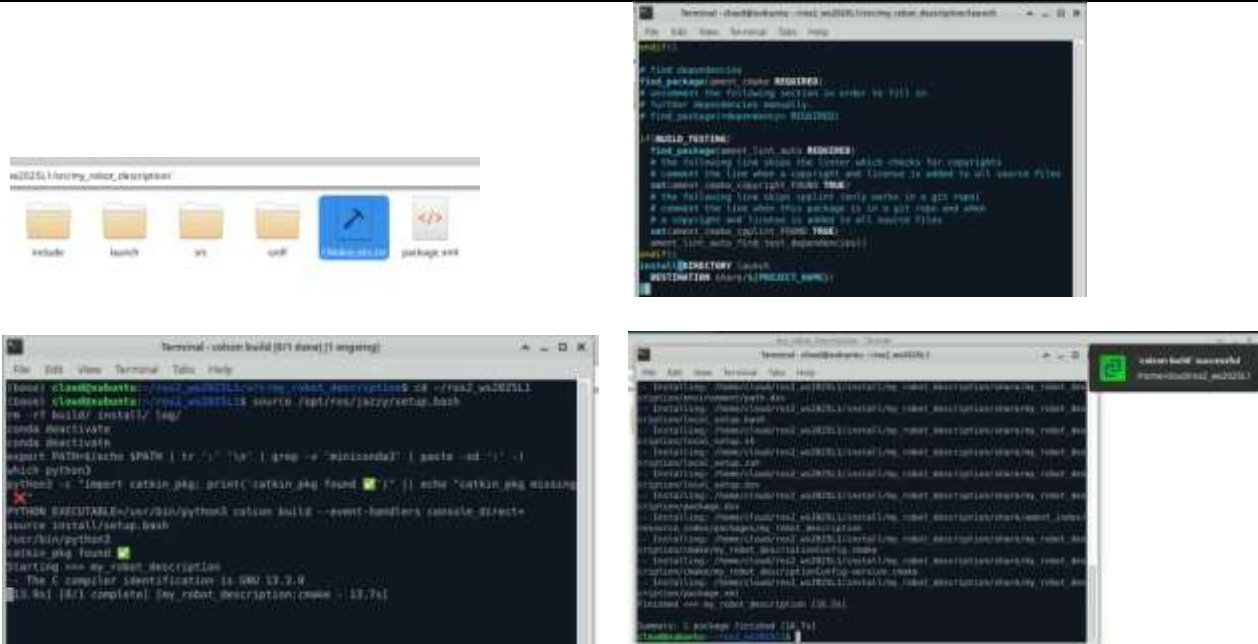


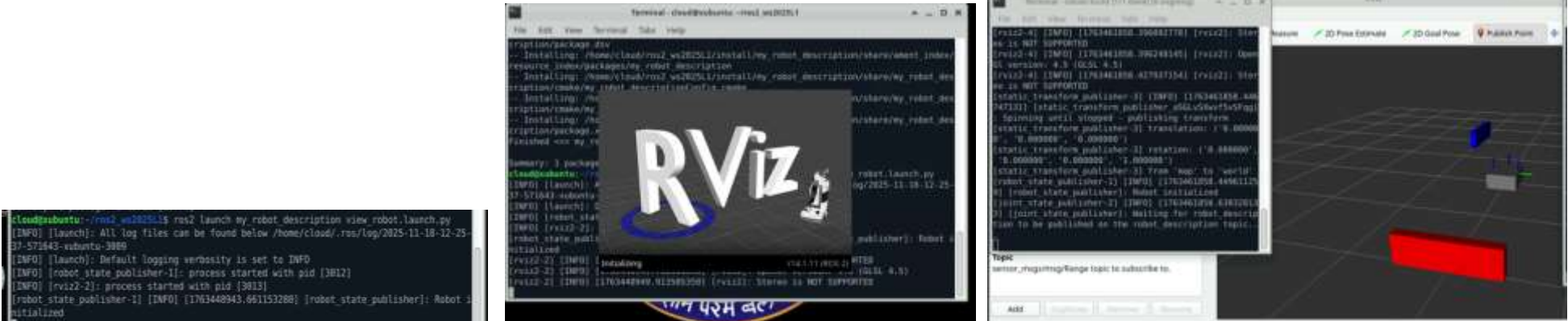
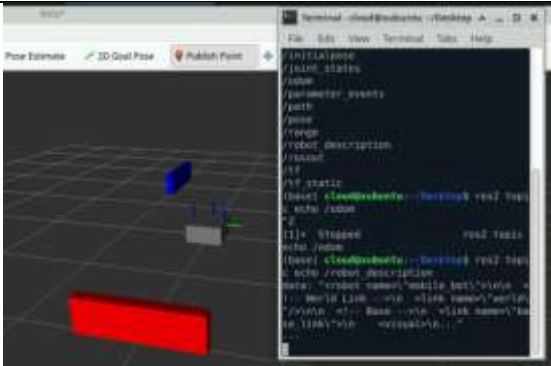
II	Design "Robot"		
2.1	Open your terminal and navigate to your package's source directory	cd my_robot_description mkdir urdf launch cd urdf	
2.2	Design of the robot description		
2.2.1	Create the file named mobile_bot.urdf	touch mobile_bot.urdf	
2.2.2	Open the file for editing	vim mobile_bot.urdf	Write/Copy the program of mobile_bot.urdf in Appendix II then save it, close the text editor window
			

2.3	Design of Controller		
	Instruction	Commands to Execute	Note
2.3.1	Ensure you are in the correct package's source directory.	cd ~/ros2_ws2025L1/src/my_robot_description	
2.3.2	Create a folder to store the controller logic and create the file motion_controller_node.py	mkdir -p scripts touch scripts/moving_bot.py chmod +x scripts/moving_bot.py cd scripts	
2.3.3	Open the File for editing	vim moving_bot.py	Write the code from in Appendix II , save it and close the text editor window

The collage consists of four screenshots arranged in a 2x2 grid. The top-left screenshot shows a terminal window with the command `cd ~/ros2_ws2025L1/src/my_robot_description`. The top-right screenshot shows a terminal window with the commands `mkdir -p scripts`, `touch scripts/moving_bot.py`, `chmod +x scripts/moving_bot.py`, and `cd scripts`. The bottom-left screenshot shows a terminal window with the code for the `moving_bot.py` file, including imports, class definition, and a main function. The bottom-right screenshot shows a terminal window with the command `vim moving_bot.py`.

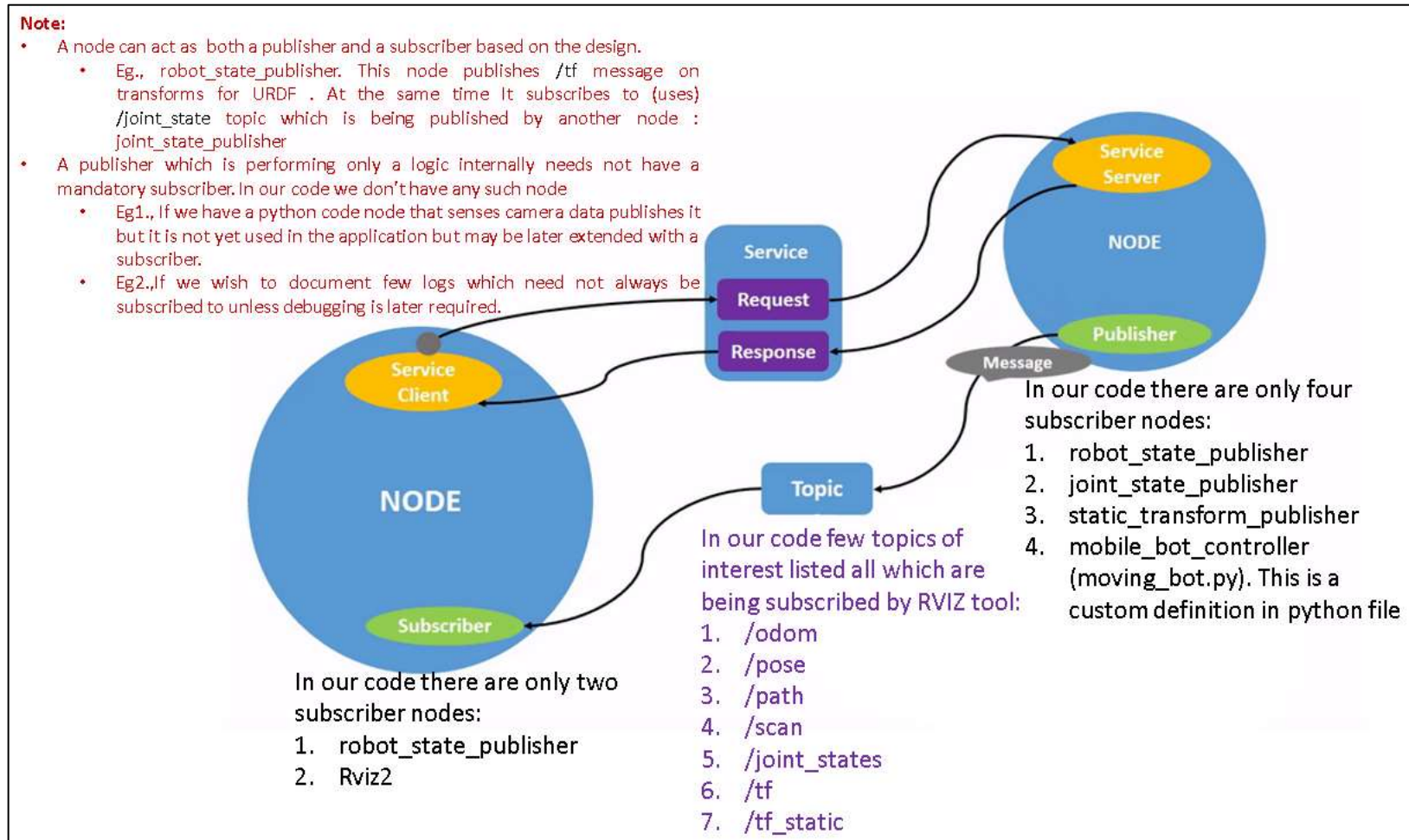
2.4	Integrate the components		
2.4.1	Navigate to your package's launch directory	cd ~/ros2_ws2025L1/src/my_robot_description/launch/	
2.4.2	Create and Open launch file for editing	touch view_mobile_bot.launch.py vim view_mobile_bot.launch.py	Write the code from in Appendix II , save it and close the text editor window
			
III	Build the designed model as ROS2 Package		
3.1	Set up the starting script for building	cd .. vim CMakeLists.txt	Append the code with existing file. Refer from in Appendix II , for the code to append, save it and close the text editor window
3.2	Navigate back to your workspace root and build	cd ~/ros2_ws2025L1	
3.3	Ensure ROS2 is running, anaconda is deactivated, Errors capture if there are any necessary package missing for the deployment	source /opt/ros/jazzy/setup.bash rm -rf build/ install/ log/ conda deactivate conda deactivate export PATH=\$(echo \$PATH tr ':' '\n' grep -v 'miniconda3' paste -sd ':') -) which python3 python3 -c "import catkin_pkg; print('catkin_pkg found ✓')" echo "catkin_pkg missing ✗" PYTHON_EXECUTABLE=/usr/bin/python3 colcon build --event-handlers console_direct+ source install/setup.bash	

3.4	Package the code	colcon build --packages-select my_robot_description	
3.5	Source your ROS2 Workspace	source install/setup.bash	
			
IV	Run & Test the Design		
4.1	In the same terminal (Terminal 1) window, the project executes by launch instruction as below. Once run, this code will automatically open RVIZ simulation tool where live robot movement is visualized. DON'T CLOSE TERMINAL from the background till the simulation is completes		
	Instruction	Commands to Execute	Note
4.2	Terminal 1		This runs the Robot Simulator
4.2.1	Run the robot simulator code	ros2 launch my_robot_description view_mobile_bot.launch.py	Observe the terminal output in the Terminal 1

			
4.3	Terminal 2		This is to test if the messages are being published in the topic
4.3.1	Open another terminal		
4.3.2	View if the URDF is correctly published. You may try to check other topics as well similar to above	ros2 topic echo /robot_description	
V	Interpretation of the result You can see the RViz window open up automatically updated with above settings. - Mobile robot's moves .It keeps sensing via LASER scan points - Upon detecting obstacle in the vicinity(defined by threshold in the python code logic), this triggers a logic which lets robot change direction - The robot to turns, rotate according to the logic of the program		

Appendix I

Figure 1: Architecture Designed in Lab 1



Appendix II

mobile_bot.urdf	moving_bot.py
Prepared by, Prof. Raja vadhana raja.vadhana@pilani.bits-pilani.ac.in Course Faculty	Updated & Reviewed by, Prof. Raja vadhana, Prof. Sangeetha raja.vadhana@pilani.bits-pilani.ac.in , sangeetha.viswanathan@pilani.bits-pilani.ac.in Course Faculty

<pre> <robot name="mobile_bot"> <!-- World Link --> <link name="world"/> <!-- Base --> <link name="base_link"> <visual> <geometry> <box size="0.5 0.4 0.2"/> </geometry> <material name="gray"> <color rgba="0.5 0.5 0.5 1"/> </material> </visual> </link> <!-- Fixed joint from world to base_link --> <joint name="world_to_base" type="fixed"> <parent link="world"/> <child link="base_link"/> <origin xyz="0 0 0.1" rpy="0 0 0"/> </joint> <!-- Left Wheel --> <link name="left_wheel"> <visual> <geometry> <cylinder radius="0.05" length="0.02"/> </geometry> <origin xyz="0 0 0" rpy="1.5708 0 0"/> <material name="black"> <color rgba="0 0 0 1"/> </material> </visual> </pre>	<pre> #!/usr/bin/env python3 import math import rclpy from rclpy.node import Node from geometry_msgs.msg import PoseStamped from nav_msgs.msg import Odometry, Path from sensor_msgs.msg import LaserScan from sensor_msgs.msg import JointState from tf2_ros import TransformBroadcaster from geometry_msgs.msg import TransformStamped from tf2_ros.static_transform_broadcaster import StaticTransformBroadcaster class MobileBotController(Node): def __init__(self): super().__init__('mobile_bot_controller') # Robot geometry self.wheel_radius = 0.05 self.wheel_separation = 0.36 # State self.x = 0.0 self.y = 0.0 self.yaw = 0.0 self.omega_l = 4.0 self.omega_r = 4.0 # Publishers self.odom_pub = self.create_publisher(Odometry, 'odom', 10) self.pose_pub = self.create_publisher(PoseStamped, 'pose', 10) self.path_pub = self.create_publisher(Path, 'path', 10) self.scan_pub = self.create_publisher(LaserScan, 'scan', 10) self.joint_pub = self.create_publisher(JointState, 'joint_states', 10) # Wheel angles for animation </pre>
---	--

```

</link>

<joint name="left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="left_wheel"/>
  <origin xyz="0 0.18 0.05" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
</joint>

<!-- Right Wheel -->
<link name="right_wheel">
  <visual>
    <geometry>
      <cylinder radius="0.05" length="0.02"/>
    </geometry>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <material name="black"/>
  </visual>
</link>

<joint name="right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="right_wheel"/>
  <origin xyz="0 -0.18 0.05" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
</joint>

<!-- Caster -->
<link name="caster_wheel">
  <visual>
    <geometry>
      <sphere radius="0.03"/>
    </geometry>
    <material name="darkgray">
      <color rgba="0.3 0.3 0.3 1"/>

```

```

self.left_angle = 0.0
self.right_angle = 0.0

# Path message
self.path_msg = Path()
self.path_msg.header.frame_id = 'world'

# TF broadcasters
self.tf_broadcaster = TransformBroadcaster(self)
self.static_broadcaster = StaticTransformBroadcaster(self)

# Publish static world->odom once
static_t = TransformStamped()
static_t.header.stamp = self.get_clock().now().to_msg()
static_t.header.frame_id = 'world'
static_t.child_frame_id = 'odom'
static_t.transform.translation.x = 0.0
static_t.transform.translation.y = 0.0
static_t.transform.translation.z = 0.0
static_t.transform.rotation.x = 0.0
static_t.transform.rotation.y = 0.0
static_t.transform.rotation.z = 0.0
static_t.transform.rotation.w = 1.0
self.static_broadcaster.sendTransform(static_t)

# Timer
self.dt = 0.05
self.timer = self.create_timer(self.dt, self.update)

# Walls
self.walls = [
  {'type': 'x_const', 'x': 1.0, 'ymin': -0.5, 'ymax': 0.5},
  {'type': 'y_const', 'y': 1.0, 'xmin': -0.5, 'xmax': 0.5},
  {'type': 'x_span_y_const', 'xmin': -1.5, 'xmax': -0.5, 'y': -0.5},
]

```

```

</material>
</visual>
</link>

<joint name="caster_joint" type="fixed">
  <parent link="base_link"/>
  <child link="caster_wheel"/>
  <origin xyz="-0.2 0 0.03" rpy="0 0 0"/>
</joint>

<!-- Wall 1 -->
<link name="wall1">
  <visual>
    <geometry>
      <box size="0.1 1 0.3"/>
    </geometry>
    <origin xyz="1 0 0.15" rpy="0 0 0"/>
    <material name="red">
      <color rgba="1 0 0 1"/>
    </material>
  </visual>
</link>

<joint name="world_to_wall1" type="fixed">
  <parent link="world"/>
  <child link="wall1"/>
  <origin xyz="1 0 0" rpy="0 0 0"/>
</joint>

<!-- Wall 2 -->
<link name="wall2">
  <visual>
    <geometry>
      <box size="0.1 1 0.3"/>
    </geometry>

```

```

def raycast(self, angle):
    cx, cy = self.x, self.y
    dx, dy = math.cos(angle), math.sin(angle)
    min_range = float('inf')

    for w in self.walls:
        if w['type'] == 'x_const':
            if abs(dx) > 1e-6:
                t = (w['x'] - cx) / dx
                if t > 0:
                    y_hit = cy + t * dy
                    if w['ymin'] <= y_hit <= w['ymax']:
                        min_range = min(min_range, t)
        elif w['type'] == 'y_const':
            if abs(dy) > 1e-6:
                t = (w['y'] - cy) / dy
                if t > 0:
                    x_hit = cx + t * dx
                    if w['xmin'] <= x_hit <= w['xmax']:
                        min_range = min(min_range, t)
        elif w['type'] == 'x_span_y_const':
            if abs(dy) > 1e-6:
                t = (w['y'] - cy) / dy
                if t > 0:
                    x_hit = cx + t * dx
                    if w['xmin'] <= x_hit <= w['xmax']:
                        min_range = min(min_range, t)

    return min_range if math.isfinite(min_range) else 10.0

def update(self):
    # Wheel velocities → odometry
    v_l = self.wheel_radius * self.omega_l
    v_r = self.wheel_radius * self.omega_r

```

<pre> <origin xyz="0 1 0.15" rpy="0 0 0"/> <material name="green"> <color rgba="0 1 0 1"/> </material> </visual> </link> <joint name="world_to_wall2" type="fixed"> <parent link="world"/> <child link="wall2"/> <origin xyz="0 1 0" rpy="0 0 0"/> </joint> <!-- Wall 3 --> <link name="wall3"> <visual> <geometry> <box size="1 0.1 0.3"/> </geometry> <origin xyz="-1 -0.5 0.15" rpy="0 0 0"/> <material name="blue"> <color rgba="0 0 1 1"/> </material> </visual> </link> <joint name="world_to_wall3" type="fixed"> <parent link="world"/> <child link="wall3"/> <origin xyz="-1 -0.5 0" rpy="0 0 0"/> </joint> </robot> </pre>	<pre> v = (v_r + v_l) / 2.0 w = (v_r - v_l) / self.wheel_separation self.yaw += w * self.dt self.x += v * math.cos(self.yaw) * self.dt self.y += v * math.sin(self.yaw) * self.dt # LaserScan simulation num_beams = 180 angle_min = -math.pi / 2 angle_max = math.pi / 2 angle_inc = (angle_max - angle_min) / num_beams ranges = [self.raycast(self.yaw + angle_min + i * angle_inc) for i in range(num_beams)] # Obstacle avoidance: check forward beam center_idx = num_beams // 2 if ranges[center_idx] <= 0.5: self.omega_l = -4.0 self.omega_r = 4.0 else: self.omega_l = 4.0 self.omega_r = 4.0 # Wheel angles (use computed v_l/v_r) omega_l = v_l / self.wheel_radius omega_r = v_r / self.wheel_radius self.left_angle += omega_l * self.dt self.right_angle += omega_r * self.dt # Publish TF t = TransformStamped() t.header.stamp = self.get_clock().now().to_msg() t.header.frame_id = 'odom' t.child_frame_id = 'base_link' </pre>
--	--

	<pre> t.transform.translation.x = self.x t.transform.translation.y = self.y qz = math.sin(self.yaw / 2.0) qw = math.cos(self.yaw / 2.0) t.transform.rotation.z = qz t.transform.rotation.w = qw self.tf_broadcaster.sendTransform(t) # Odometry odom = Odometry() odom.header = t.header odom.header.frame_id = 'odom' odom.child_frame_id = 'base_link' odom.pose.pose.position.x = self.x odom.pose.pose.position.y = self.y odom.pose.pose.orientation.z = qz odom.pose.pose.orientation.w = qw odom.twist.twist.linear.x = v odom.twist.twist.angular.z = w self.odom_pub.publish(odom) # PoseStamped pose = PoseStamped() pose.header = t.header pose.header.frame_id = 'world' pose.pose.position.x = self.x pose.pose.position.y = self.y pose.pose.orientation.z = qz pose.pose.orientation.w = qw self.pose_pub.publish(pose) # Path self.path_msg.header.stamp = t.header.stamp self.path_msg.poses.append(pose) self.path_pub.publish(self.path_msg) </pre>
--	---

	<pre> # LaserScan scan = LaserScan() scan.header = t.header scan.header.frame_id = 'base_link' scan.angle_min = angle_min scan.angle_max = angle_max scan.angle_increment = angle_inc scan.range_min = 0.02 scan.range_max = 10.0 scan.ranges = ranges self.scan_pub.publish(scan) # JointStates js = JointState() js.header = t.header js.name = ['left_wheel_joint', 'right_wheel_joint'] js.position = [self.left_angle, self.right_angle] self.joint_pub.publish(js) def main(): rclpy.init() node = MobileBotController() rclpy.spin(node) node.destroy_node() rclpy.shutdown() if __name__ == '__main__': main() </pre>
<u>view mobile_bot.launch.py</u>	<u>CMakeLists.txt (Code must be appended to existing file)</u>
<pre> from launch import LaunchDescription from launch_ros.actions import Node def generate_launch_description(): </pre>	<pre> install(DIRECTORY launch DESTINATION share/\${PROJECT_NAME}/) install(PROGRAMS scripts/moving_bot.py DESTINATION lib/\${PROJECT_NAME} </pre>

```

urdf_path =
'/home/cloud/ros2_ws2025L1/src/my_robot_description/urdf/mobile_bot.urdf'

return LaunchDescription([
    Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': open(urdf_path).read()}],
        output='screen'
    ),
    Node(
        package='joint_state_publisher',
        executable='joint_state_publisher',
        name='joint_state_publisher',
        output='screen'
    ),
    Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        arguments=['0', '0', '0', '0', '0', '0', 'map', 'world']
    ),
    Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen'
    ),
    Node(
        package='my_robot_description',
        executable='moving_bot.py',
        name='mobile_bot_controller',
        output='screen'
    )
])

```

```
)
```

Self-Practice Exercises for the students

1. In the given above code, locomotion of robot will not terminate. Edit the necessary code to let it run only for few iterations or for few seconds.
2. Try to build additional controller actuators. Eg., Modify existing wheels as front . Add two more wheels as rear. Update the project files accordingly.
3. Instead of odometry motion model, implement velocity motion model. Tip : Check the function **update()** in the controller file **moving_bot.py**
4. Synthetically generate reliable data to predict the velocities (for all the wheels) given the current pose and sensor reading (rays). Here the velocities is the target(Y) and the data from (pose, sensor, etc.,) is the feature (X). Train using DNN/ML model. Replace step 3 with the learnt model for motion control
5. Implement correlation based sensor model to correct the pose. Refer to our pybullet based experiment demo'd python file in webinar for reference. Tip: Remember to impute Gaussian noise in the pose before this experimentation to experience the effect of the correction step.
6. Critically assess redesign requirements if the controller is modified as a reinforcement learner, which needs to learn a policy for locomotion. Formulate MDP, list the nature of state space, action space, check which clause of DRL (Deep Reinforcement Learning) algorithm is best applicable in this use case.